数据结构 · 链表算法题（上）· 考点精讲

408 数据结构大题专项 ｜ 课程第 2 讲（分 P3） ｜ 时长 28 分 21 秒

本讲范围：承接上一讲「顺序表算法题」，只做了四件事 —— ① 快速过一遍链表的五种形态；② 逐字讲清**结点定义**（老师说「考试肯定是要自己写的」）；③ 讲一段教材上没有、他自己加的「补充知识点」：**链表 ↔ 数组互相转换**，作为考场兜底写法；④ 开始讲第 17 题「求倒数第 k 个结点」，刚说完思路，本讲结束。
读法建议：§2.2 的结点定义必须能默写；§3、§4 是本讲唯一给了完整代码的内容，照着敲三遍；§5 是这讲唯一涉及的真题（2009 · 42），它讲完在分 P4。

目录

1 考情分析：本讲说了什么、没说什么
2 核心概念：五种形态与结点定义
3 解题方法论：万金油兜底法
4 典型题型与模板（两份完整代码）
5 统考真题精解：2009 · 42 求倒数第 k 个结点
6 易错点清单
7 考场速查卡
附 本资料的来源与边界

ASR 校正说明

转写稿由本地 Whisper `small` 模型在 CPU 上生成，术语同音错字密度很高。本资料引用时统一校正，主要替换：数组（速度）、双链表（装链表）、头结点（投机点）、尾指针（为止针）、负一（附一）、箭头 `->`（电投）、struct（strike）、LNode（lnote）、typedef（type df）、ElemType（Elimen Type）、int（印它）、char（制服）、树（差数）、快慢指针（宽慢指针）、开销（开箱）、赋值（复制）、仿写（防写）、cur（COR/COL）。**凡不能确定的，一律不写进正文，集中列在 §8。**

1 考情分析：本讲说了什么、没说什么

老师本讲原话	含义
「就是 15 分你可能得个 9 分 10 分」	链表算法题按 **15 分**一道大题算；这句是他在评价「转成数组」这个兜底解法能拿到的分数
「这块代码考试肯定是要自己写的，包括后面学到的树（转写「差数」）也是有这样的代码」	结点定义是算法题的**前置必写内容**，不是可选项
「要区分清楚 LNode* 和 LinkList 两个的区别」	他认为这是链表代码里最容易写含糊的地方，讲了两遍

本讲没有的内容（不要凭本讲推断）

本讲**没有**给链表算法题的历年频次统计，也**没有**点名任何真题年份。老师只说「做一下 17」——「17」是他手上题单的编号，不是统考年份。凡涉年份与频次的说法都出自 §1.2 与 §5 引用的公开真题资料，**不是老师口径**。

1.1 老师给的学习路径

> 「书上这边有很多代码，课下的话可以看一看……最开始我就直接讲那个题目就行，基本就足够了。讲完题目之后，把讲过的题目你写个三四遍，到时候再去看书上这些应该都是能看懂的。」「这个课下一定要对着上课讲过的（代码）**仿写**一下，因为上课写的这个代码一般质量都还算可以的。」

推进顺序：**结点定义 → 补充知识点（链表 ↔ 数组）→ 讲义上的题目**；书上源码他只让课下自己看。

1.2 补充：链表算法题在统考中的出现情况

下表**不是老师讲的**，来自公开真题资料的整理，用来看这一节的题量级；引用请以真题原卷为准。

年份	题目	与本讲的关系
2009	查找单链表中倒数第 k 个位置上的结点（15 分）	**本讲的第 17 题就是它**，见 §5
2012	求两个带头结点单链表公共后缀的起始位置	本讲未涉及
2015	删除单链表中绝对值相等的元素（空间换时间）	本讲未涉及
2019	重排链表 $L'=(a_1, a_n, a_2, a_{n-1}, \ldots)$，要求空间 O(1)	本讲未涉及

2 核心概念：五种形态与结点定义

2.1 五种链表形态

老师翻到书上第 30 页，用一分半钟把这五种形态的判定特征过了一遍：「常见的链表结构简单回忆一下……单链表、双链表、循环单链表、循环双链表，这些东西可以快速回忆一下。」

形态	老师给的判定特征（校正后原话）	老师额外提醒
单链表	一个指针指向下一个结点	—
双链表	「两个指针，一个指针往后指，一个往前指」	「第一个指针前面是没有东西的，所以它前面也是空」（头结点前驱为空）；「具体后面写代码还是有一些东西要注意的」——**本讲未展开**
循环单链表	「最后一个结点，它在指向第一个」	—
循环双链表	「第一个往前指向最后一个，然后最后一个往后指向第一个」	—
静态链表	「用一个数组来表示一个链表」：每个分量存数据，另存**下一个结点的下标**，最后一个结点存 `-1`	「一般（这种）都是用数组……链表是一种链式存储，静态链表就是用一个数组来表示一个链表」

静态链表的例（下标照录）：「A 是在 5 号位置，B 是在 0 号位置，C 是在 3 号位置。那这个 A 的后面是 B，那（就在 A 里）存一个 B 的下标…… C 后面是最后一个，存一个（-1）。」

2.2 结点定义：这一块必须能默写

老师说「这块代码考试肯定是要自己写的」，随后把书上第 30 页那段代码一个词一个词拆开讲。按他逐句念出的内容还原（**讲稿无板书，还原说明见 §4.3**）：

```
typedef struct LNode {      /* struct：结构体关键字，表示自己定义的类型 */
    ElemType data;          /* 抽象类型：题目里是 int 就写 int，是 char 就写 char */
    struct LNode *next;     /* 结构体指针：指向下一个结点 */
} LNode, *LinkList;         /* 两个别名：LNode = struct LNode；LinkList = struct LNode * */
```

- `ElemType` **是抽象类型，不是真类型。**「并没有一个类型叫 ElemType，你平常用的时候，如果题目里具体是 int，你这里就写 int，是 char 就写 char，它（只是）表示万能的、普通的基本类型。」
- **结点里为什么要有一个指向自己的指针。**「每一个链表结点，它是有一个数值，还会有一个指针指向下一个结点，所以这个需要自己去定义。」「现在是一个 `LNode *`，那说明这个指针是指向一个链表的结点。」
- `typedef` **起两个别名。**「正常来说结构体并不是基本类型，不像 int 你直接用就行。要用一个结构体，你必须加上 `struct` 这个字，比如定义一个变量 a 必须这样去用，用起来就很繁琐。这里就起了一个别名：把 `struct LNode` 起名叫 `LNode`；第二步，把 `struct LNode` 的星 —— 这是一个整体 —— 起名叫 `LinkList`。」

> **老师当场强调的书写细节**
>
> 「一定要加上分号，结构体的语法要加上分号。」`}` 后那个 `;` 是便宜丢分点。

2.3 `LNode *` 与 `LinkList`：等价，但含义不同

> 「正常来说一个链表，每一个结点你都可以看成一个 `LNode *`。但是**第一个结点往往比较特殊，第一个结点它可以表示整条链表**，所以把第一个结点又起了一个名字叫 `LinkList`。」「本质上 `LinkList` 完全等价于 `LNode *`。你也可以把后面的结点叫成 `LinkList`，理论上可以这样写，但是这样就不太符合它最初的定义了。」

写法	读代码时的含义	老师原话
`LNode *`	操作**单个结点**：插入、删除、修改某一个结点	「如果你传进来的参数是一个 `LNode *`，你就知道你现在操作的是一个单个结点，把某一个结点比如插入、删除、修改。」
`LinkList`	操作**整条链表**：例如把整个链表反转	「如果你这个函数传进来的是 `LinkList`，这样你就知道是对整个链表进行操作，比如把整个链表进行反转。」

> **为什么第一个结点能代表整条链表**
>
> 「因为链表的话就是拿第一个结点表示整条链表，**只要有了第一个结点，就能找到后面的所有的**。」所以第一个结点特殊，单独起名 `LinkList`；后面的结点都只是 `LNode *`。老师也承认二者可混用：「实际上是等价的，但是后者往往表达意思更直观。」

2.4 头结点：本讲所有代码的前提

> 「注意这个（链表题）里面往往都是带**头结点**的。链表都会带头结点，就最开始……比如说链表有四个结点 1、2、3、4，往往会在最前面给它加一个头结点，头结点里面不去存数据，就是加个头结点后面方便操作的。」

- 头结点不存数据，遍历一律从 `head->next` 起步：「因为 head 是头结点，头结点后面才是实际的第一个结点。」
- 头指针变量名老师习惯叫 `head`：「一般头结点起名就叫 head。」

3 解题方法论：万金油兜底法

3.1 老师自己加的「补充知识点」：链表 ↔ 数组

他翻到书上第 44 页说，这一节的书上**没有**专门讲这个，是他自己补的：

> 「先来看一个链表的补充知识点：**链表的问题有时候可以转化成数组去做**，这样开销很大但是比较万能，这个可以学一下。书上虽然没有专门讲，这边可以练习一下。第一个是把链表存到数组，然后还有一个是把数组构造成链表。」

> **什么时候用（老师给的使用条件）**
>
> **只在没有别的办法时用。**「（这是）比较万金油的写法、比较万能的写法，很多情况都能用，但是可能开销会大点，就是 15 分你可能得个 9 分 10 分。但是总比（空着强）—— 实在没有别的办法，你可以用这种。」「没有其他思路，或者其他思路不会写，你可以用这种做法。」

使用前提 —— 必须先知道表长：「因为如果长度都不知道，你辅助数组都没法开的。」「但如果题目里没有告诉你长度，你可以自己去求一下链表的长度，待会我们有一道题会专门在讲。」⚠️ 这道「专门讲求长度」的题，**本讲没讲到**（见 §8）。

方向	步骤（按老师口述的顺序整理）
链表 → 数组	① 确定表长 `n`，开 `n+1` 个元素的辅助数组；② `p` 从头结点的下一个结点起步（`p = head->next`）；③ `for (i = 1; i <= n; i++)`：把 `p->data` 存进 `A[i]`，然后 `p` 后移一步。
数组 → 链表	① 先 `new` 一个头结点 `head`（不存数据）；② 定义 `cur` 同样指向 `head`，充当**尾指针**；③ 循环 `n` 次：每次 `new` 一个结点 `node`，把 `A[i]` 赋进 `node->data`，把 `node` 接到 `cur` 后面，再让 `cur` 后移一步；④ 返回 `head`。

> **老师说这一步为什么必须另设 cur**
>
> 「所以这里为什么要格外定一个 `cur`？因为 `cur` 会不停地往后走，充当一个**尾指针**的作用。这个 `head` 的话始终在头，最终你要把 `head` 返回。」—— 让 `head` 自己往后走，链表就找不回头了。

3.2 第二条路径：双指针 / 快慢指针

> 「（第 17 题）昨天我们讲了双指针、快慢指针可以做。」

老师把它当**首选**（「昨天我们讲了……可以做」），但本讲不再展开 —— 它在上一讲「顺序表算法题」里讲过。使用示范见 §5.2。

4 典型题型与模板（两份完整代码）

本讲唯一给出完整代码的就是这两个转换函数。老师念代码时没有板书，下面是按他逐句口述的操作顺序还原的版本，**变量名以他念出的为准**（`head` / `cur` / `node` / `p`），函数名是整理时补的。

4.1 模板 A：把链表存到数组

```
/* 把带头结点单链表 head 中的元素依次存入数组 A（A 已开好 n+1 个元素） */
void ListToArray(LinkList head, int A[], int n) {
    LNode *p = head->next;              /* 头结点不存数据，从第一个实际结点开始 */
    for (int i = 1; i <= n; i++) {      /* 0 号下标不用，从 1 号开始对齐链表的第 1 个元素 */
        A[i] = p->data;                 /* 通过箭头访问结点的数据域 */
        p = p->next;                    /* 当前结点存完，p 往后走一步 */
    }
}
```

- **参数为什么写 `LinkList`**：「写成 `LinkList` 就知道传进来的是一个链表；写成 `LNode *` 你就以为传进来的是一个结点，虽然两者等价，但是这个意思就更直观了。」
- **为什么从 `head->next` 开始**：「因为 head 是头结点，头结点后面才是实际的第一个结点。」
- **为什么 0 号下标空着不用**：「数组的 0 号下标我不用，从 1 号（开始），这样对应刚好第一个就是链表的第一个。那这个数组就要开 `n+1` 个了，因为 0 号下标不用，就把它浪费掉。」
- **为什么用 `for`**：「感觉写成 `for` 循环要更好一点，因为这个数组的下标是从 1 开始的。」「总共循环刚好执行 n 次，刚好能把每一个元素都放在数组里面。」

4.2 模板 B：把数组构造成链表

```
/* 由数组 A[1..n] 构造一条带头结点的单链表，返回头指针 */
LinkList ArrayToList(int A[], int n) {
    LNode *head = new LNode;        /* 先申请头结点，不存数据，名字一般叫 head */
    head->next = NULL;              /* ← 讲稿未念到这一行，见 4.3 的还原说明 */
    LNode *cur = head;              /* cur 也指向头结点，之后充当尾指针 */
    for (int i = 1; i <= n; i++) {  /* 循环执行 n 次，每次创建一个结点 */
        LNode *node = new LNode;    /* 局部变量：每轮循环都新开辟一个结点 */
        node->data = A[i];          /* 结构体指针用箭头修改成员：把 A[i] 放进去 */
        cur->next = node;           /* 把新结点接到 cur 后面 —— 这一步必须手动写 */
        cur = cur->next;            /* cur 后移一步，仍指向当前链表最后一个结点 */
    }
    return head;                    /* 返回头结点，head 就代表整条链表 */
}
```

- **返回值为什么写 `LinkList`**：「比如说现在有返回值，所以我返回值写一个 `LinkList`，你就知道返回的不是一个结点，是一整条链表，就把第一个结点返回。」
- **`new LNode` 是什么**：「把数组构造成链表，肯定要先在内存上开辟一个空间……这个叫从内存上申请一个 `LNode` 的变量，然后返回的是一个指针。」
- **局部变量 `node` 每轮重建**：「因为这个 `node` 是局部变量，所以第二次又开了一个结点，新结点也叫 `node`，（把值）放进去，把 `node` 再放在 `cur` 后面，接下来让 `cur` 再往后走。」
- **结构体指针必须用箭头**：「你要通过箭头把它里面的元素进行修改。」
- **最后一个结点不用手动置空**：「最后一个结点你刚创建好，它这个指针域应该是空，这个你不用再去手动给它赋值了。但是如果你想要非空，比如说第一个结点后面想要接第二个结点，你需要再去手动赋值一下。」

> **本讲最实用的一条细节**
>
> 「因为每一个结点刚创建的时候，它这个指针域默认就是空，数值域默认是零……如果是 int 类型这个数，刚创建默认就是零，指针默认是空，不需要再去管。」

4.3 代码还原的边界（必读）

> ⚠ **这些是整理时补的，不是老师念的**
>
> ① **函数名**（`ListToArray` / `ArrayToList`）与部分参数名：老师只说「写一个函数 FUNC」。
> ② `head->next = NULL;` 这一行**讲稿里没有**；按老师「新建即默认空」的说法可不写，显式写上更稳，故保留并标出。
> ③ 循环边界 `i = 1; i <= n` 是依据老师「0 号不用 / 从 1 号开始 / 共循环 n 次」三点还原的，他口述时没有逐字念三段式。
> ④ 开数组那一句老师念的是 `malloc`，构造结点那一句念的是 `new LNode` —— **两种风格并存是老师原话的样子**，不是整理时混用的；考试时统一成一种即可，但别抄混。

5 统考真题精解：2009 · 42 求倒数第 k 个结点

> **这道题是怎么认定出来的（先看依据）**
>
> 老师在 27 分 35 秒只说：「做一下 17。」「这个题目它让求的倒数第 k 个结点。」——**没说年份，也没说分值**。本资料把题目与统考卷做了内容比对：**2009 年 408 数据结构第 42 题**的题面正是「带头结点的单链表、只给头指针、查找倒数第 k 个位置上的结点」，与老师描述吻合，故按该题呈现。**题面与评分说明引自公开真题资料，不是老师讲的**；这个比对结论也请你对着原卷核一遍。

5.1 题面（2009 · 42，15 分）

> 已知一个带有表头结点的单链表，结点结构为 `| data | link |`。假设该链表只给出了头指针 `list`。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点（k 为正整数）。若查找成功，算法输出该结点的 data 域的值，并返回 1；否则，只返回 0。要求：（1）描述算法的基本设计思想；（2）描述算法的详细实现步骤；（3）根据设计思想和实现步骤，采用程序设计语言描述算法（使用 C、C++ 或 Java 语言实现），关键之处请给出简要注释。

注意它的三段式：**思想 → 步骤 → 代码**。官方评分说明里还有一句关键的话：「算法程序题，如果能够写出数据结构类型定义，正确的算法思想都会至少给一半以上分数。」—— 这正印证 §1 里老师说的「结点定义考试肯定是要自己写的」。

5.2 解法一：双指针（快慢指针）—— 老师首选

老师本讲说到的：「昨天我们讲了双指针、快慢指针可以做。」

具体做法（2009 年官方设计思想，不是老师本讲的话）：两个指针 p、q 初始均指向头结点的下一个结点；p 沿链表移动，当 p 移到第 k 个结点时，q 开始与 p 同步移动；当 p 移到最后一个结点时，q 所指即倒数第 k 个结点。**只扫描一遍。**

```
/* 2009 · 42 参考实现（公开题解；函数名与局部变量名为整理时所加） */
int FindKthFromEnd(LNode *head, int k) {
    LNode *p = head->next;      /* p、q 均从第一个实际结点出发 */
    LNode *q = head->next;
    int count = 0;
    while (p != NULL) {         /* 一趟扫描 */
        if (count < k) {
            count++;            /* 前 k 步：只有 p 在走 */
        } else {
            q = q->next;        /* k 步之后：q 与 p 同步走 */
        }
        p = p->next;
    }
    if (count < k) return 0;    /* k 超过表长，查找失败 */
    printf("%d\n", q->data);
    return 1;
}
```

5.3 解法二：转成数组 —— 老师给的兜底法

> 「如果你用刚才我们讲的那种数组的方法，你看它这种开销大点也是能做：你把整个链表放到数组里面，最后把数组（对应位置）返回这个倒数第 k 个就可以了。」

⚠️ 老师在这里只说思路，**没有写代码**，转写随即结束。下面是按 §4 模板 A 补写的版本，用来对照他的思路 —— **代码不是老师念的**。

```
/* 老师思路的补写版：求长度 → 存数组 → 取 A[n-k+1] */
int FindKthFromEnd_Array(LinkList head, int k) {
    int n = 0;
    LNode *p = head->next;
    while (p != NULL) { n++; p = p->next; }        /* 第一遍：求表长 */
    if (k <= 0 || k > n) return 0;                 /* k 非法或超过表长 */
    int *A = (int *)malloc(sizeof(int) * (n + 1)); /* 0 号下标不用，开 n+1 个 */
    p = head->next;
    for (int i = 1; i <= n; i++) {                 /* 第二遍：填数组 */
        A[i] = p->data;
        p = p->next;
    }
    printf("%d\n", A[n - k + 1]);                  /* 倒数第 k 个 = 正数第 n-k+1 个 */
    free(A);
    return 1;
}
```

5.4 这道题的得分上限：官方评分说明

你的解法	说明里的上限	和老师那句对得上吗
一遍扫描得到正确结果	**15 分**	老师按 15 分算这道题 ✓
采用两遍或多遍扫描 / 递归 / 空间复杂度过高（用了与 k 或表长有关的辅助数组）	最高 10 分	老师估「15 分你能得个 9 分 10 分」—— 与官方「最高 10 分」是同一条界线 ✓

这张表引自公开真题解析，**请以官方答案为准**。它说明一件事：老师的兜底法不是偏方，而是一条**明确能给到 10 分**的保底路线 —— 所以「实在不会就转数组」是划算的。

6 易错点清单

易错写法 / 想法	为什么错	正确写法	出处
`} LNode, *LinkList` 后不写分号	结构体定义是完整语句，缺分号编译不过	「一定要加上分号，结构体的语法要加上分号」	老师原话
要处理整条链表，参数却写 `LNode *`	两者等价但语义不直观，读代码的人（包括你自己）会误解	整条链表写 `LinkList`；只操作一个结点才写 `LNode *`	老师原话
遍历从 `head` 本身开始，把 `head->data` 也算进去	头结点不存数据，会多算一个元素、整体错位	一律 `p = head->next;` 起步	老师原话
结构体指针写 `p.data`	指针类型必须用箭头解引用	`p->data`、`p->next`	老师原话
辅助数组从下标 0 存起，忘了与链表第 1 个元素对齐	取「倒数第 k 个」时下标换算容易差一位	0 号下标空着不用，从 1 号存，数组开 `n+1` 个	老师做法
不知道表长就开数组	「长度都不知道，你辅助数组都没法开的」	先遍历一遍求长度 `n`，再开数组	老师原话
构造链表时让 `head` 自己往后走，或忘记 `cur = cur->next;`	前者走完找不回头结点；后者 `cur` 不动，下一轮会把结点接在同一位置，链表断掉	另设 `cur` 当尾指针；每轮末尾让 `cur` 后移一步	老师原话
手动给最后一个结点写 `next = NULL`	老师认为这步多余：「最后一个结点你刚创建好，它这个指针域应该是空」	不管最后一个结点；但中间结点必须手动 `cur->next = node;`	老师原话
空指针写成 `null` / `Null`	老师转述的要求是「（空指针）应该是全大写」	`NULL`（⚠ 原句转写为「全大解」，推断为「全大写」，见 §8）	推断

教材 / 语言层面的补充（不是老师原话）

① C 语言里结构体内部那个自引用指针**必须写成** `struct LNode *next` —— 此时别名 `LNode` 还没生效，写成 `LNode *next` 在 C 下编译不过（C++ 可以）。

② C++ 的 `new LNode;` 不带括号时成员**未初始化**，只有 `new LNode();` 才置零 —— 考场上显式写 `node->next = NULL;` 更保险（与老师「新建即空」不冲突，只是更稳）。

③ 「先接后断」：在 `p` 之后插入 `s`，必须先 `s->next = p->next;` 再 `p->next = s;`，写反会丢掉后继链。本讲没有讲到插入，这条是教材内容。

7 考场速查卡

```
【一】结点定义（必须默写，末尾分号不能丢）
typedef struct LNode {
    ElemType data;          // 题目是 int 就写 int
    struct LNode *next;     // 结构体内必须带 struct
} LNode, *LinkList;
//  LNode   = 一个结点        LinkList = 整条链表（第一个结点 / 头结点）

【二】写函数签名时的判断规则
参数、返回值是 LinkList   →  操作整条链表（反转、构造、遍历）
参数、返回值是 LNode *    →  操作单个结点（插入、删除、修改）

【三】链表 → 数组（兜底法①）
p = head->next;
for (i = 1; i <= n; i++) { A[i] = p->data; p = p->next; }
// 0 号下标空着不用，数组开 n+1 个；n 未知就先求长度

【四】数组 → 链表（兜底法②）
head = new LNode;  cur = head;            // cur 当尾指针
for (i = 1; i <= n; i++) {
    node = new LNode;  node->data = A[i];
    cur->next = node;  cur = cur->next;   // 先接后移，两句都不能少
}
return head;                              // 返回 head，不是 cur

【五】两句话记住得失
转数组：开销大但万能，15 分的题约拿 9~10 分，实在没思路时用它。
双指针：一趟扫描、O(1) 空间，满分路线 —— 但代码本讲没讲。
写代码前先确认三件事：① 带不带头结点？② 表长已知吗？③ 要求空间 O(1) 吗？
```

如果只记三件事

① 结点定义必须能默写，包括那个分号；② `LinkList` = 整条链表、`LNode *` = 单个结点，用函数签名表达你的意图；③ 链表 ↔ 数组是能拿 10 分的兜底路线，但要记得「0 号下标不用」和「`cur` 是尾指针」这两个坑。

附：本资料的来源与边界

来源	内容	可靠性与边界
B 站《408 各科大题知识点强化课》第 2 讲「数据结构 · 链表算法题」分 P3，音频 **28 分 21 秒**（1700.79 s）	§1 考情（仅老师口径）、§2、§3、§4 全部，§5.2/§5.3 的老师思路，§6 老师部分	本地 Whisper `small` 模型 CPU 转写，讲稿 428 行、约 1.7 万字节，**覆盖** `[00:00:13]`–`[00:28:00]`，即**全程**。错字密度高，引用处已校正并注明。这一段**没有板书**，§4 与 §5.3 的代码是按他逐句口述的命令顺序还原的。
公开真题资料（2009 年 408 第 42 题题面与官方评分说明）	§1.2 年份表、§5.1 题面、§5.2 官方设计思想与参考实现、§5.4 评分说明	**不是老师讲的**，用于把老师那句「做一下 17」对应到统考卷上。评分说明请以官方答案为准。
教材 / 语言通识	§6 末尾那个补充框	三条都不是老师本讲的内容，已标在框内。

⚠ **转写含糊、按上下文推断的地方（逐条列出）**

① `[00:15:15]`「链表里面的空应该是全大解」→ 推断为「空指针**应该是全大写**」（即 `NULL`）；「全大解」无法直译，这是最可能的读法。

② `[00:18:36]`「但是总比那个控制墙」→ 疑为「总比**空着强**」，语义通但不确证，本资料未把它当原话用在别处。

③ `[00:19:54]`「叫 6 Lnode」与 `[00:20:45]`「有 newlnode」→ 推断为 `new LNode`。⚠ 老师前面开数组时确实念了 `malloc`，所以这里也可能是在口述 `malloc(sizeof(LNode))`；两种读法在考场上都成立，本资料按 `new` 记录。

④ `[00:27:52]`「也可以存到复古」→「复古」是什么无法确定（可能是「辅助（数组）」或其他被听错的词），**本资料没有采信，正文未写入该句。**

⑤ `[00:16:10]`「你要通过这个电投」→ 推断为「**箭头**（`->`）」，与后文「通过箭头把它里面的元素进行修改」一致。

⑥ `[00:02:01]`「双链表……挂起来是比较简单的」→ 疑为「**画起来**」（画图），已在 §2.1 标注。

⑦ `[00:10:38]`「然后呃后这边是四道题目啊」→ 语序混乱，只能推断为「书上有几道题目」，**「四」这个数字不敢用**，正文因此没写题量。

⑧ **循环边界与数组下标**：`for (i = 1; i <= n; i++)` 是按老师「0 号不用 / 从 1 号开始 / 共循环 n 次」三点还原的，他口述时没逐字念三段式。

⑨ **函数名**（`ListToArray` / `ArrayToList` / `FindKthFromEnd`）与部分参数名是整理时添加的。

⑩ 「书上第 30 页」（结点定义）、「书上第 44 页」（补充知识点）—— 页码照录，但**讲稿没说这是哪一本书**，本资料因此没写书名。

老师没讲完 / 本讲没有覆盖的内容

① **第 17 题的代码没讲**：老师说完「双指针、快慢指针可以做」「数组法也能做」之后，转写就结束了。

② **双指针 / 快慢指针的具体写法**：老师说「昨天我们讲了」，属上一讲内容，本讲未复习。

③ **求链表长度的函数**：老师说「待会我们有一道题会专门在讲」，本讲没讲到。

④ **双链表的代码细节**：老师说「具体后面写代码还是有一些东西要注意的」，本讲未展开。

⑤ **教材上的其余题目**，以及书上第 44 页那两道题的完整代码（本讲只讲了转换模板）。

⑥ **本讲的后续**：链表这一讲被切成两段 —— 分 P4「链表算法题（2）」还有 **2 小时 23 分 56 秒**（8635.87 s），第 17 题及后面的题都在那里，**不在本资料范围内**。这也是本讲只有 28 分钟的原因：它不是完整的一讲，而是「链表算法题」的开头一段。

整合自 28 分 21 秒网课讲稿（本地 Whisper 转写，无板书，代码按口述还原）+ 2009 年真题公开材料 · 未上传任何材料到第三方服务 · 生成于 2026-10-10